

OS Deadlock & Resource Allocation Android App

AI Build Roadmap / Project Specification

Purpose: Build an educational Android application that interactively demonstrates Deadlock Detection and Banker's Algorithm. The AI/developer should implement the project in phases, keeping algorithmic logic independent from Android UI so it can be tested reliably.

1. Project Understanding

The application is an Operating Systems learning/simulation tool. Users enter process/resource data, choose an algorithm, run it, and receive an understandable result. The project is not intended to be a complex production Android application; correctness of the OS algorithms and clarity of the simulation are the primary goals.

2. Target User Flow

Screen	Purpose
Home	Explain the project and provide navigation to Deadlock Detection and Banker's Algorithm.
Deadlock Detection	Collect allocation, request, and available-resource data; validate input; run detection.
Banker's Algorithm	Collect allocation, maximum-demand, and available-resource data; calculate Need and safety.
Results	Show safe/unsafe or deadlock/no-deadlock status, relevant matrices, and sequence/reasoning.

3. Recommended Technical Stack

- **Language:** Kotlin
- **IDE:** Android Studio
- **UI:** XML layouts + Activities (keep the implementation simple and beginner-friendly unless the existing project explicitly uses another UI framework).
- **Architecture:** UI → View/Controller layer → pure Kotlin algorithm layer → Result model → Results UI.
- **Testing:** Kotlin/JUnit unit tests for algorithms; Android emulator/manual tests for UI.
- **Version control:** Git + GitHub, with feature branches where useful.

4. Core Algorithm Requirements

4.1 Deadlock Detection

Implement the standard deadlock-detection procedure using Allocation, Request, and Available. Start $Work = Available$. Processes whose Request can be satisfied by Work can finish; then release their Allocation back into Work. Processes that cannot finish after the procedure are deadlocked.

Inputs: number of processes P, number of resource types R, Allocation[P][R], Request[P][R], Available[R].

Output: deadlock status, finished processes, deadlocked processes (if any), and enough intermediate information to explain the result.

4.2 Banker's Algorithm

Implement the standard safety algorithm. Calculate $Need = Max - Allocation$. Work starts as Available. Find an unfinished process whose $Need \leq Work$, simulate completion, add its Allocation to Work, and continue. If every process can finish, return SAFE with the safe sequence; otherwise return UNSAFE.

Inputs: number of processes P, number of resource types R, Allocation[P][R], Max[P][R], Available[R].

Output: Need matrix, safe/unsafe status, safe sequence if one exists, and useful intermediate steps.

5. Data Model

Do not pass raw strings through the entire application. Convert validated UI input into typed Kotlin models. Suggested conceptual models:

ProcessInput → processCount, resourceCount, allocationMatrix, requestMatrix/maxMatrix, availableVector

AlgorithmResult → status, sequence, deadlockedProcesses, needMatrix, workSteps, messages/errors as appropriate

Keep algorithm functions deterministic and independent of Activities, Views, EditTexts, or Android Context.

6. Phase-Based Implementation Roadmap

Phase 0	Project setup	Create Android Studio project, verify emulator, establish Git repo, confirm build works.
Phase 1	Pure Kotlin algorithms	Implement Deadlock Detection and Banker's Algorithm without Android UI. Add unit tests and console/test data.
Phase 2	UI skeleton	Build Home, Deadlock Detection, Banker's, and Results screens with simple XML layouts and navigation.
Phase 3	Integration	Connect form input → validation → typed data model → algorithm → result model → Results screen.
Phase 4	Validation	Handle empty/non-numeric/negative values, inconsistent dimensions, $Max < Allocation$, $Request > Available$, and invalid resource counts.
Phase 5	Visualization	Show matrices, Need calculation, safe sequence/deadlocked processes, and optional step-by-step Work updates.
Phase 6	Testing	Test safe, unsafe, deadlock, no-deadlock, boundary, and invalid-input cases on both logic and UI.
Phase 7	Polish	Improve spacing, labels, readability, error messages, reset buttons, and navigation only after correctness is confirmed.
Phase 8	Documentation & demo	Prepare README, screenshots, algorithm explanation, test cases, architecture diagram, and viva/demo flow.

7. UI Requirements

- **Home:** project title, short explanation, two algorithm buttons/cards.
- **Input screens:** process count and resource count; generate or display matrix input fields dynamically.
- **Deadlock screen:** Allocation, Request, Available inputs and Run button.
- **Banker's screen:** Allocation, Max, Available inputs and Run button.
- **Results:** prominent status, matrices, sequence/deadlocked processes, and explanation.
- **Usability:** labels must clearly identify rows/processes and columns/resources; avoid unexplained abbreviations.
- **Reset:** allow users to clear and enter another scenario.

8. Input Validation Rules

- Process count and resource count must be positive integers and should have a reasonable UI limit.
- All matrix/vector entries must be integers ≥ 0 .
- Allocation cannot exceed Max for Banker's Algorithm.
- Banker's Need values must never be negative.
- A request/demand must not exceed the currently available resources when the relevant algorithm requires that condition.
- Every matrix must have exactly P rows and R columns.
- Do not crash on malformed input; show a clear user-facing error.

9. Testing Matrix

Category	Cases
Banker's	Known safe state; known unsafe state; single process; one resource type; all allocations zero.
Deadlock	Known deadlock; no deadlock; all processes finish; blocked processes remain.
Validation	Blank field; text; negative; decimal; mismatched dimensions; Max < Allocation; zero processes/resources.
UI	Navigation; keyboard/input; scrolling for larger matrices; reset; repeated runs; rotation/state behavior if supported.

10. AI Implementation Instructions

- **Work incrementally.** Do not generate the entire application in one step.
- **Inspect the current repository first.** Never overwrite existing working code without understanding it.
- **Build Phase 1 before Phase 2.** Algorithm correctness must be established independently.
- **Explain assumptions.** If requirements are ambiguous, choose the simplest reasonable implementation and state the assumption.
- **Prefer simple Android concepts.** Avoid unnecessary libraries, frameworks, networking, databases, authentication, or cloud services unless explicitly requested.
- **Keep logic testable.** Algorithms must not depend on Android classes.
- **Run tests/build after meaningful changes.** Fix compilation and test failures before moving forward.
- **Do not fake results.** Results displayed by the UI must come from the implemented algorithms.
- **Preserve educational value.** The final result should make it possible for a student to explain why a state is safe, unsafe, deadlocked, or not deadlocked.

11. Suggested Project Structure

A simple structure is preferred. Exact package names can be adapted to the existing project.

```
app/ src/main/java/.../ MainActivity.kt DeadlockActivity.kt BankerActivity.kt ResultsActivity.kt
algorithms/ DeadlockDetector.kt BankersAlgorithm.kt models/ AlgorithmInput.kt AlgorithmResult.kt
validation/ InputValidator.kt utils/ MatrixParser.kt src/main/res/layout/ activity_main.xml
activity_deadlock.xml activity_banker.xml activity_results.xml src/test/.../
DeadlockDetectorTest.kt BankersAlgorithmTest.kt
```

12. Definition of Done

- The project builds successfully in Android Studio.
- The app runs on an emulator/device.
- Both algorithms produce correct results for known test cases.
- Input is validated and invalid input does not crash the app.
- The UI can dynamically represent the selected number of processes/resources.
- Results are clearly understandable to an OS student.
- Unit tests cover the core algorithms and important edge cases.
- The code is separated into UI, models, validation, and pure algorithm logic.
- README/documentation explains the problem, algorithms, architecture, setup, and test cases.

13. Recommended Build Order for an AI Coding Agent

1. Inspect repository/project configuration.
2. Create/verify the Kotlin algorithm package.
3. Implement Banker's Algorithm + tests.
4. Implement Deadlock Detection + tests.
5. Run all tests and correct algorithm issues.
6. Create Android screens and navigation.
7. Create dynamic matrix input components.
8. Implement parsing and validation.
9. Connect each screen to the corresponding algorithm.
10. Build the Results screen with status + matrices + sequence/process information.
11. Test the complete app using fixed known scenarios.
12. Add polish and documentation.

14. Non-Goals

- No backend/server is required.
- No database is required.
- No login/authentication is required.
- No internet connection should be necessary for the core app.
- Do not add complex architecture solely to make the project look advanced.
- Do not prioritize animations or visual polish over algorithm correctness.

Final instruction to the AI: Treat this document as the project's implementation contract. First determine the current project state, then execute the roadmap phase-by-phase. At the end of every phase, verify the result before proceeding. Keep the implementation understandable enough that the student can explain every important part during an OS viva.